GitHub essentials for Docking Universal

A practical guide for scientific users who are new to GitHub

What this guide is for
This guide covers the parts of GitHub that matter when obtaining, updating, citing, troubleshooting, or contributing to Docking Universal. No prior GitHub experience is assumed, but routine command-line familiarity is.

What GitHub is doing here

1. **Distributing the software**
   Clone the repository for an updateable working copy, or download a ZIP for a one-time snapshot.
2. **Identifying exact software states**
   A release or commit identifier provides a precise reference for computational provenance.
3. **Tracking improvements and problems**
   Issues document reproducible problems; pull requests contain proposed changes under review.
4. **Keeping development separate from studies**
   The repository is software. Study inputs, protocol bundles, outputs, and reports belong in managed research folders.

Using Docking Universal does not require contributing code. The central workflow is simply: obtain a known software version, install it, record that version with the study, and update deliberately rather than automatically during an active analysis.

Recommended path

Use pages 2-5 for routine use. Pages 6-7 cover public issues and optional contributions; page 8 is a command reference.

1. GitHub and Git are not the same thing

The names sound similar, but they refer to different parts of the process.

Term	Plain-language meaning	Why you might care
GitHub	A website that stores and displays projects.	This is where you view Docking Universal and download it.
Git	A program that keeps a history of changes to files.	It can download the project and later retrieve updates.
Repository	The project folder and its recorded history. People often shorten this to *repo*.	The Docking Universal repository contains its code, documentation, and tests.
Clone	Make a complete local copy of a repository.	Use this when installing with Git and when you want easy updates.
Commit	A named, recorded checkpoint in the file history.	Mostly relevant if you change the project files.
Branch	A separate line of work that does not immediately change the main version.	Useful when proposing a contribution safely.
Pull request	A request on GitHub to review and combine a branch into the main project.	Use this if you want maintainers to consider your change.

A helpful distinction
GitHub stores the shared project online. Your computer holds a separate local copy. Changes on one side do not automatically appear on the other.

What you need

A GitHub account is not required to view or clone this public repository. Git is needed for the recommended clone-and-update workflow; Conda is needed for the scientific environment. Forks, rebases, Actions, and merge conflicts are not required for routine use.

2. Obtain the repository

Cloning is recommended because it preserves the project history and provides a controlled update path. A ZIP is appropriate when you need only a one-time snapshot.

Option A: Download a ZIP file

On the repository page, choose **Code > Download ZIP**, then expand the archive. This produces a normal project folder but does not retain Git history or an update connection.

> **Important**
> A ZIP download is a snapshot. It does not include an easy built-in way to retrieve later updates. You can always download a newer ZIP and install that version separately.

Option B: Clone with Git - recommended

Copy the HTTPS address from the repository's **Code** menu, choose a location for software projects, and clone the repository:

```
mkdir -p ~/Projects
cd ~/Projects
```

```
git clone https://github.com/thomaslane79-creator/docking-universal.git
cd docking-universal
```

The resulting folder contains the current files and the recorded history needed by `git pull`. Keep this software folder separate from study inputs and outputs.

3. Install and confirm the command

These steps happen inside the Docking Universal folder. Conda creates an isolated environment so the scientific packages do not overwrite unrelated software on your computer.

First installation

1. **Create the scientific environment**
   This reads the project's environment file and installs the listed scientific packages. It may take several minutes.

```
conda env create -f environment.yml
```

2. **Activate the environment**
   Activation tells the terminal which isolated collection of programs to use.

```
conda activate docking-universal
```

3. **Install the Docking Universal command**
   This places the command inside the active environment so it works from any folder while that environment is active.

```
make install-conda
```

4. **Check the installation**
   The first command shows where the program is installed. The second displays its command list.

```
which docking-universal
docking-universal --help
```

5. **Create the Vina engine environment**
   Docking Universal finds this environment automatically when docking is requested.

```
conda env create -f environments/vina.yml
```

> **Each new Terminal window**
> Before using Docking Universal, run `conda activate docking-universal`. You can then run `docking-universal` from any directory.

A sensible first check

```
docking-universal check-install
docking-universal validate quick
```

The validation check confirms that the program can see its required tools. It does not validate a new biological target or guarantee docking accuracy.

4. Keep software and research data distinct

The repository is the software source. Your receptor files, ligand libraries, protocols, and results are research records. Keeping them separate makes both updating and archiving safer.

Software folder	Study folder
A cloned or downloaded Docking Universal repository	Inputs, approved .duprotocol bundles, docking outputs, reports, and analysis notes
Can be replaced or updated from GitHub	Should be backed up according to your laboratory's data-management plan
Avoid placing study results here	Use meaningful target, ligand-set, and date identifiers

Retrieve an update

If you cloned the project and have not edited its files, return to the repository folder and run:

```
cd ~/Projects/docking-universal
git pull
conda activate docking-universal
make install-conda
```

The pull step brings the newest project history from GitHub into your local copy. Reinstalling updates the command inside the Conda environment.

> **Before updating during an active study**
> Record the current Docking Universal and dependency versions, preserve the approved .duprotocol bundle, and avoid mixing results from different software versions without documenting the comparison. Reproducibility matters more than always using the newest code.

If Git says your local changes would be overwritten

Stop and do not force the update. Git is protecting files that differ from the shared version. Ask a colleague or project maintainer to help preserve or review those changes.

5. How to read a GitHub project page

Most scientists need only a few parts of the page.

1. **README**
   The project overview. Start here for purpose, scientific scope, limitations, and the command summary.

2. **docs**
   The manual. Installation, validation, guided workflows, methodology, and interpretation details live here.

3. **Issues**
   A public place to report a reproducible problem, ask a focused project question, or suggest an improvement.

4. **Releases**
   Named project versions, when releases are published. A release is easier to cite and reproduce than an unspecified moving branch.

5. **Pull requests**
   Proposed changes under review. Ordinary users can usually ignore this tab.

Before reporting a problem

- Remove confidential, unpublished, patient-derived, or access-controlled data unless sharing is explicitly permitted.
- Run `docking-universal check-install` and note which required tool is missing or failing.
- Record your operating system, computer architecture, Docking Universal version, and the exact command used.
- Describe what you expected and what actually happened. Include the smallest safe example that reproduces the problem.
- Paste text errors as text when practical. Screenshots are useful for visual problems but are harder to search.

> **Public means public**
> Assume that anything posted in a GitHub issue can be read and copied by anyone. Use synthetic or public structures when demonstrating a problem.

What the project history can establish

A commit identifies a particular state of the software. A release or commit identifier can therefore support computational provenance. It does not, by itself, establish that a protocol is scientifically valid for a target or that two environments are equivalent.

6. Optional: suggest a change

You do not need to contribute code to use the software. If you do want to correct documentation or propose an improvement, GitHub keeps that work separate until it is reviewed.

The basic contribution path

1. **Create or choose a branch**
   A branch is a safe line of work separate from the main version.

```
git switch -c your-name/short-description
```

2. **Edit and inspect the files**
   Make one focused change. Do not include study data, credentials, or unrelated files.

3. **See what changed**
   Git lists edited or newly created files. This does not publish anything.

```
git status
git diff
```

4. **Record a checkpoint**
   Add only the intended files, then create a short descriptive commit.

```
git add path/to/file
git commit -m "Explain the scientific or documentation change"
```

5. **Push the branch**
   This sends the branch to GitHub. It still does not alter the main project.

```
git push -u origin your-name/short-description
```

6. **Open a pull request**
   GitHub normally displays a button after the push. Describe what changed, why, and how you checked it.

> **Nothing silently becomes official**
> A pushed branch is a proposal. A maintainer reviews the pull request before deciding whether it should become part of the main project.

If you only want to report a problem

Open an issue instead. You do not need to create a branch, edit code, or understand pull requests.

7. One-page reference

What you want to do	What to use
Get a simple one-time copy	GitHub: Code > Download ZIP
Get a copy that can be updated	`git clone WEB_ADDRESS`
Enter the project folder	`cd docking-universal`
Activate the software environment	`conda activate docking-universal`
Install the public command	`make install-conda`
Confirm the command is available	`which docking-universal`
See Docking Universal commands	`docking-universal --help`
Check scientific dependencies	`docking-universal check-install`
Retrieve updates	`git pull`, then `make install-conda`
Report a reproducible problem	Open a GitHub issue; do not include restricted data
Propose a project change	Branch > commit > push > pull request

The safe default
If you are unsure, stop before deleting files, forcing a Git operation, or posting research data. Your cloned software copy can be replaced. Your scientific records may not be recoverable.

Where to go next

Use the Docking Universal README for scientific scope and command selection. Use the installation guide for platform-specific dependencies. Use the guided-workflow manual before beginning a control-backed or ligand-free docking study.

This guide explains project access and version-control basics only. It is not a docking protocol and does not replace scientific validation, target-specific controls, or local data-governance requirements.